

TERMINALE NSI

Algorithmique

La Recherche Textuelle

Algorithme Naïf & Boyer-Moore

Niveau	Terminale NSI -- conforme au programme officiel
Theme	Algorithmique -- Recherche dans un texte
Competences visees	Comprendre, implementer et comparer des algorithmes de recherche textuelle
Prerequis	Chaines de caracteres (indexation, len), boucles for/while, fonctions Python

Document concu pour les eleves de niveau intermediaire a faible -- explications detaillees et progressives

Introduction : Pourquoi chercher dans un texte ?

Imaginez que vous avez un livre de 500 pages et que vous cherchez le mot **algorithme**. Vous ne lisez pas tout depuis le début ! Vous utilisez une stratégie. Un ordinateur fait pareil : il existe plusieurs méthodes, plus ou moins rapides, pour trouver un mot dans un grand texte.

En NSI Terminale, on étudie deux algorithmes : **l'algorithme naïf (simple mais lent)** et **l'algorithme de Boyer-Moore (plus complexe mais très rapide)**.

Vocabulaire essentiel

Texte (T)	La grande chaîne de caractères dans laquelle on cherche. Sa longueur est notée n .
Motif (M)	Le petit mot qu'on veut trouver dans le texte. Sa longueur est notée m .
Indice	La position d'un caractère dans une chaîne. En Python, les indices commencent à 0.
Occurrence	Un endroit où le motif a été trouvé dans le texte.

Exemple : dans le texte **"ABRACADABRA"** ($n = 11$), si on cherche le motif **"ABR"** ($m = 3$), on trouvera deux occurrences : aux indices 0 et 7.

Rappel Python : les chaînes de caractères

Avant d'étudier les algorithmes, voici les notions Python indispensables :

```
texte = "BABAY"
motif = "BAY"

# Longueur d'une chaîne
n = len(texte)    # n = 5
m = len(motif)    # m = 3

# Accès à un caractère par son indice (commence à 0)
texte[0] --> 'B'
texte[1] --> 'A'
texte[4] --> 'Y'

# Comparaison de caractères
texte[0] == motif[0] --> True    (B == B)
texte[2] == motif[2] --> False   (B != Y)
```

Partie 1 - L'Algorithme Naif (Force Brute)

1.1 L'idée generale

L'algorithme naif est la methode la plus intuitive. Il simule ce que ferait quelqu'un qui cherche un mot manuellement dans un texte :

1. On place le motif au-dessus du debut du texte (indice 0).
2. On compare les caracteres un par un, de gauche a droite.
3. Si tous les caracteres correspondent, on a trouve le motif !
4. Si un caractere ne correspond pas, on decale le motif d'une seule case vers la droite.
5. On recommence jusqu'a avoir essaye toutes les positions possibles.

Analogie du monde reel

C'est comme chercher un mot dans un livre en lisant toutes les pages depuis le debut. Vous regardez chaque lettre, et des qu'une lettre ne correspond pas, vous avancez d'une case. Simple, mais long pour de grands textes !

1.2 Exemple detaille pas-a-pas

Cherchons le motif **"BAY"** (m=3) dans le texte **"BABAY"** (n=5) :

Etape	Alignement	Comparaisons (gauche -> droite)	Resultat
1	B A B A Y / B A Y	B==B OK A==A OK B!=Y ECHEC	Echec. Decalage de 1.
2	B A B A Y / _ B A Y	A!=B ECHEC (des le 1er caractere)	Echec immediat. Decalage de 1.
3	B A B A Y / _ _ B A Y	B==B OK A==A OK Y==Y OK	MOTIF TROUVE a l'indice 2 !

Dans cet exemple, le motif est trouve apres seulement 3 etapes. Mais si le texte etait tres long, cela pourrait prendre beaucoup plus de temps...

1.3 Le code Python - Version complete commentee

Voici le code Python tel qu'il peut etre demande en epreuve pratique. Lisez chaque commentaire attentivement :

```
def recherche_naive(motif, texte):
    """Recherche la premiere occurrence du motif dans le texte.
    Renvoie l'indice de debut si trouve, -1 sinon."""
    n = len(texte)    # longueur du texte
    m = len(motif)    # longueur du motif

    # On essaie chaque position possible pour le debut du motif.
```

```

# Pourquoi n - m + 1 ?
# Le motif ne peut pas commencer apres l'indice n-m
# (il n'y aurait plus assez de place pour le motif entier).
for i in range(n - m + 1):
    j = 0    # j parcourt le motif de la gauche

    # On compare lettre par lettre
    while j < m and texte[i + j] == motif[j]:
        j = j + 1    # Les lettres correspondent, on avance

    # Si j a atteint m, c'est qu'on a tout lu sans erreur
    if j == m:
        return i    # On renvoie la position de debut

return -1    # Motif non trouve dans tout le texte

```

Comprendre range(n - m + 1)

Exemple : texte = "BABAY" (n=5), motif = "BAY" (m=3)

$n - m + 1 = 5 - 3 + 1 = 3 \rightarrow \text{range}(3)$ donne les indices 0, 1, 2

Si on essayait l'indice 3, il faudrait comparer texte[3], texte[4], texte[5]... mais texte[5] n'existe pas ! C'est pour ca qu'on s'arrete a n-m.

1.4 Complexite et limites

La **complexite** d'un algorithme mesure le nombre d'operations qu'il effectue en fonction de la taille des donnees.

Meilleur cas	Le motif est trouve tout de suite (en position 0). Tres rapide.
Pire cas	Exemple : motif = "AAAB", texte = "AAAAAAAAAA". A chaque position, on compare presque tout le motif avant de detecter l'echec. Complexite : $O(n \times m)$
Cas moyen	En pratique avec des textes naturels (français, anglais...), les echecs sont detectes rapidement. La complexite est proche de $O(n)$.

Limite principale : Sur de tres grands textes (ADN, gros fichiers), la complexite $O(n \times m)$ devient trop lente. C'est la qu'intervient Boyer-Moore.

Partie 2 - L'Algorithme de Boyer-Moore

2.1 Les deux idées révolutionnaires

L'algorithme de Boyer-Moore (1977) est bien plus rapide dans la pratique. Il repose sur deux changements fondamentaux par rapport à l'algorithme naïf :

Idee 1 - Lecture de droite à gauche	Idee 2 - Sauts intelligents
Au lieu de comparer les caractères du motif de gauche à droite, Boyer-Moore commence par la DERNIERE lettre du motif. Cela peut sembler bizarre, mais c'est beaucoup plus efficace !	En cas d'erreur, au lieu de décaler d'une seule case (comme l'algo naïf), Boyer-Moore calcule un saut qui peut être bien plus grand, évitant ainsi de tester des positions inutiles.

2.2 L'heuristique du mauvais caractère

L'heuristique (règle pratique) du mauvais caractère est la stratégie utilisée pour calculer les sauts. Voici comment elle fonctionne :

La règle en 2 cas	
Cas 1 : La lettre du texte qui a causé l'erreur N'EXISTE PAS dans le motif	On fait sauter le motif entièrement au-delà de cette lettre. Saut = longueur du motif (m).
Cas 2 : La lettre du texte qui a causé l'erreur EXISTE dans le motif	On aligne le motif pour que sa dernière occurrence de cette lettre corresponde à la lettre du texte. Le saut est calculé grâce à la table des sauts.

2.3 Exemple détaillé pas-à-pas

Cherchons le motif "**CHIEN**" (m=5) dans le texte "**UN CHIEN**" (n=8) :

Etape	Alignement	Comparaison (droite -> gauche)	Décision
1	UN _ CHIEN / CHIEN	On commence par la dernière lettre : texte[4] = 'H' != motif[4] = 'N' -> Erreur ! Le mauvais caractère est 'H'.	'H' existe dans le motif à l'indice 1. On calcule le saut pour aligner les 'H'.
2	UN _ CHIEN / ___ CHIEN	N==N OK E==E OK I==I OK H==H OK C==C OK Tout correspond !	MOTIF TROUVE à l'indice 3 !

Avec l'algorithme naïf, on aurait effectué plus de comparaisons. Boyer-Moore a trouvé le motif en seulement 2 étapes grâce au saut intelligent !

2.4 Le prétraitement : construction de la table des sauts

Avant de commencer la recherche, Boyer-Moore analyse le motif et construit un **dictionnaire de sauts**. Ce dictionnaire indique, pour chaque lettre du motif, de combien de cases décaler en cas d'échec.

Formule du saut pour la lettre a l'indice i :

$$\text{saut} = \text{longueur_motif} - 1 - \text{indice_de_la_lettre}$$

Attention : on ne traite PAS la dernière lettre du motif (indice m-1), car elle serait la lettre comparée en premier dans Boyer-Moore et ne peut pas provoquer un saut utile.

Exemple avec le motif **"CHIEN"** (m = 5) :

Indice i	0	1	2	3	4	Remarque
Lettre	C	H	I	E	N	
Saut	4	3	2	1	--	Le 'N' (indice 4) n'est pas dans la table.

Calcul : C -> 5-1-0 = 4, H -> 5-1-1 = 3, I -> 5-1-2 = 2, E -> 5-1-3 = 1

Le code Python de la table des sauts

```
def creer_table_sauts(motif):
    """Cree le dictionnaire des sauts pour l'algorithme Boyer-Moore."""
    table = {} # Dictionnaire vide
    m = len(motif)

    # On parcourt toutes les lettres SAUF la dernière (m-1)
    for i in range(m - 1):
        lettre = motif[i]
        # Le saut = distance entre la lettre et la fin du motif
        table[lettre] = m - 1 - i

    return table # On retourne le dictionnaire

# Test :
# creer_table_sauts("CHIEN") --> {'C': 4, 'H': 3, 'I': 2, 'E': 1}
```

2.5 Le code Python complet de Boyer-Moore

Voici la version complete et commentee de l'algorithme Boyer-Moore :

```
def boyer_moore(motif, texte):
    """Recherche le motif dans le texte avec Boyer-Moore.
    Renvoie l'indice de debut si trouve, -1 sinon."""
    n = len(texte)
    m = len(motif)

    # Cas particulier : motif vide
    if m == 0:
        return 0

    # Etape 1 : Pretraitement - on cree la table des sauts
    table = creer_table_sauts(motif)

    # Etape 2 : Recherche dans le texte
    i = 0    # i = position du debut du motif sur le texte

    while i <= n - m:
        j = m - 1    # j commence a la FIN du motif (droite a gauche !)

        # On recule tant que les lettres sont identiques
        while j >= 0 and motif[j] == texte[i + j]:
            j = j - 1

        # Si j est negatif, on a tout compare avec succes
        if j < 0:
            return i    # Motif trouve a la position i

        # Sinon, on calcule le saut grace au mauvais caractere
        lettre_erreur = texte[i + j]

        # table.get(lettre_erreur, m) signifie :
        # - Si la lettre est dans la table -> on prend son saut
        # - Sinon (lettre absente du motif) -> on saute de m cases
        saut = table.get(lettre_erreur, m)
        i = i + saut    # On deplace le motif

    return -1    # Motif non trouve
```

2.6 Trace d'execution commentee

Reprenons notre exemple : motif = **"BAY"**, texte = **"BABAY"**

Table des sauts : `creer_table_sauts("BAY") = {'B': 2, 'A': 1}` (m=3, Y non inclus)

i	Alignement	j part de	Comparaison (droite -> gauche)	Saut calcule
0	BABAY / BAY	j=2	texte[2]='B' != motif[2]='Y' Mauvais caractere = 'B'	table.get('B',3) = 2 i = 0 + 2 = 2
2	BABAY / -- BAY	j=2	texte[4]='Y'==motif[2]='Y' OK texte[3]='A'==motif[1]='A' OK texte[2]='B'==motif[0]='B' OK j=-1 -> TROUVE !	Retourne i=2

Boyer-Moore a trouve le motif en seulement 2 iterations la ou l'algorithme naif en avait besoin de 3. Sur de grands textes, cette difference devient enorme !

Partie 3 - Comparaison et Bilan

3.1 Tableau comparatif

Critere	Algorithme Naif	Boyer-Moore
Sens de comparaison	Gauche -> Droite	Droite -> Gauche
En cas d'erreur	Decale de 1 case	Saut calcule (peut etre grand)
Pretraitement	Aucun	Construction table des sauts
Complexite pire cas	$O(n \times m)$ -- lent	$O(n \times m)$ -- mais rare
Complexite pratique	$O(n)$ en moyenne	Souvent meilleur que $O(n)$
Facilite de comprehension	Tres simple a comprendre	Necessite d'apprendre la table
Usage recommande	Petits textes, motifs courts	Grands textes (ADN, logs, etc.)

3.2 Retenir la complexite

Qu'est-ce que $O(n \times m)$?

Si le texte a $n=1000$ caracteres et le motif $m=10$ caracteres, dans le pire cas l'algorithme naif fait $1000 \times 10 = 10\,000$ comparaisons.

Boyer-Moore en pratique

Grace aux sauts, Boyer-Moore peut parfois trouver le motif en lisant moins de n caracteres ! C'est le seul algorithme de recherche textuelle dit sous-lineaire.

3.3 Applications reelles

Ces algorithmes ne sont pas que theoriques. On les retrouve dans de nombreux outils du quotidien :

- Fonction Ctrl+F dans les editeurs de texte et navigateurs
- Moteurs de recherche (recherche dans des pages web)
- Bioinformatique : recherche de sequences genetiques dans l'ADN
- Antivirus : recherche de signatures de virus dans des fichiers
- Commande grep dans les systemes Unix/Linux
- Correcteurs orthographiques et outils de traitement du langage naturel

Partie 4 - Exercices d'Application

Exercice 1 - Algorithme Naïf : Trace d'exécution

Enonce

Soit texte = "ABABCD" (n=6) et motif = "ABC" (m=3).

- Combien de positions i va-t-on tester ? (Calculez $n - m + 1$)
- Completez le tableau de trace ci-dessous :

i	Alignement	Comparaisons	Resultat
0	ABABCD / ABC		
1	ABABCD / _ ABC		
2	ABABCD / __ ABC		
3	ABABCD / ___ ABC		

- Quelle est la valeur retournee par recherche_naive("ABC", "ABABCD") ? Justifiez.

Exercice 2 - Construction de la table des sauts

Enonce

Pour le motif "LAPIN" (m=5), completez la table des sauts.

Rappel : saut = $m - 1 - i$, et on ne traite PAS la derniere lettre.

Lettre	L	A	P	I	N
Indice i	0	1	2	3	4
Saut					--

Que retourne alors creer_table_sauts("LAPIN") ? Ecrivez le dictionnaire Python.

Exercice 3 - Question de cours (type bac)

Questions

- Quelle est la principale difference entre l'algorithme naïf et Boyer-Moore concernant le sens de lecture du motif ?
- Expliquez ce que signifie $O(n \times m)$ pour la complexite de l'algorithme naïf.
- Dans quel cas l'algorithme Boyer-Moore effectue-t-il un saut de m cases ?
- Citez deux applications concretes des algorithmes de recherche textuelle.

Partie 5 - Corrigé des Exercices

Corrigé - Exercice 1

1. $n - m + 1 = 6 - 3 + 1 = 4$. On teste donc 4 positions : $i = 0, 1, 2, 3$.

i	Alignement	Comparaisons	Résultat
0	ABABCD / ABC	A==A OK B==B OK A!=C ECHEC	Echec, decalage +1
1	ABABCD / _ABC	B!=A ECHEC	Echec, decalage +1
2	ABABCD / __ABC	A==A OK B==B OK C==C OK	TROUVE ! i=2

3. La fonction retourne **2**, car le motif "ABC" commence à l'indice 2 dans "ABABCD".

Corrigé - Exercice 2

Lettre	L	A	P	I	N
Indice i	0	1	2	3	4
Saut	4	3	2	1	--

Le dictionnaire retourne : {'L': 4, 'A': 3, 'P': 2, 'I': 1}

Corrigé - Exercice 3

6. L'algorithme naïf compare les lettres du motif de GAUCHE à DROITE, tandis que Boyer-Moore les compare de DROITE à GAUCHE (en commençant par la dernière lettre du motif).
7. $O(n \times m)$ signifie que dans le pire des cas, le nombre d'opérations est proportionnel au produit de la longueur du texte (n) par la longueur du motif (m). Pour $n=1000$ et $m=10$, on peut faire jusqu'à 10 000 comparaisons.
8. Boyer-Moore effectue un saut de m cases lorsque le mauvais caractère (la lettre du texte qui a causé l'erreur) n'apparaît PAS dans le motif. On utilise `table.get(lettre, m)` ou m est la valeur par défaut.
9. Deux applications : le raccourci Ctrl+F dans les éditeurs/navigateurs, et la recherche de séquences génétiques en bioinformatique (ADN). On peut aussi citer les antivirus ou la commande `grep`.

Fiche Recapitulative - A memoriser pour l'epreuve

LES POINTS CLES

Algorithme Naif

- Comparaisons : gauche -> droite
- En cas d'erreur : decalage de 1 seule case
- Boucle : `for i in range(n - m + 1)`
- Complexite : $O(n \times m)$ dans le pire cas

Boyer-Moore

- Comparaisons : droite -> gauche (commence par la FIN du motif)
- En cas d'erreur : saut calcule grace a la table des sauts
- Formule saut : $m - 1 - i$ (pour la lettre a l'indice i du motif)
- Si mauvais caractere absent du motif -> saut = m
- Tres rapide en pratique sur de grands textes

Valeurs de retour communes aux deux algorithmes

- Retourne l'indice de debut si le motif est trouve
- Retourne -1 si le motif n'est pas dans le texte

Conseils pour l'epreuve pratique

- Pour l'algorithme naif : pensez a bien justifier le `range(n - m + 1)` si on vous le demande.
- Pour Boyer-Moore : commencez toujours par ecrire la table des sauts avant d'effectuer la trace.
- Si on vous demande de comparer les deux : privilegiez l'algo naif pour sa simplicite si le texte est court, et Boyer-Moore pour sa rapidite sur les grands textes.